

Universidade Federal do Maranhão
Bacharelado em Ciência da Computação
Arquitetura de Software (2026.3) – 1ª avaliação

Prof. MSc.: Lucas Reis Abreu

Discente: _____

1. De acordo com os mitos administrativos da Engenharia de Software e a conhecida "**Lei de Brooks**", acrescentar mais programadores a um projeto de software que já está atrasado nos prazos o deixará ainda mais atrasado. Explique detalhadamente por que essa afirmação tende a ser verdadeira, focando no impacto que os novos membros causam no tempo de desenvolvimento produtivo da equipe original.
2. A classificação proposta por **Bertrand Meyer** divide os sistemas de software em três categorias: A (Acute), B (Business) e C (Casuais). Recentemente, surgiu uma abordagem de desenvolvimento informal baseada em IA denominada de "**vibe coding**", na qual o programador descreve as demandas que deseja e a inteligência artificial escreve o código final. Explique por que essa abordagem pode ser tolerada em sistemas do tipo C, mas representa um risco *arquitetural* inaceitável caso seja aplicada no desenvolvimento de sistemas do tipo A e B.
3. Uma das principais vantagens apontadas para a adoção de uma arquitetura de Microsserviços é a autonomia dos times de desenvolvimento.
 - a. Por que colocar uma modificação rapidamente em *deploy* é muito mais arriscado e dependente de testes exaustivos em uma arquitetura monolítica do que em uma arquitetura de *microservies*? Explique.
 - b. Por que é considerado uma má prática que diferentes *microservies* compartilhem e acessem diretamente o mesmo banco de dados central?

4. O padrão arquitetural MVC (*Model-View-Controller*) surgiu na década de 70 para interfaces gráficas tradicionais e foi amplamente adaptado para a *web* a partir dos anos 2000. Qual é a diferença fundamental na forma como a interface é gerada e atualizada em uma aplicação *web* MVC (onde o servidor gera o *HTML*) em comparação com as *Single Page Applications* (SPAs), que são implementadas no front-end utilizando *frameworks* de JavaScript (como *React* ou *Vue*)?
5. A independência funcional é medida por dois critérios principais: coesão e acoplamento. Suponha que, na arquitetura de um sistema corporativo, um desenvolvedor criou um único módulo chamado **GerenciadorGeral**. Esse componente acumula três responsabilidades distintas: extrair e calcular os dados de um relatório financeiro, desenhar a interface gráfica com o usuário na tela e realizar a impressão física do documento. Com base nos conceitos de independência funcional, avalie o nível de coesão desse componente e explique como você redesenharia essa estrutura para que o sistema apresente *alta coesão e baixo acoplamento*.
6. Analise as duas classes em Python abaixo, que fazem parte do sistema logístico de um e-commerce:

```
class ServicoDeFrete:
    def calcular_frete(self, peso_kg: float) -> float:
        # aceita pacotes de qualquer peso até 50 kg
        pass

class FreteExpresso(ServicoDeFrete):
    def calcular_frete(self, peso_kg: float) -> float:
        # a malha expressa só aceita pacotes de até 5 kg
        if peso_kg > 5:
            raise ValueError("O peso excede o limite permitido para o frete expresso.")
        # Lógica de cálculo expresso...
        pass
```

Se o módulo de fechamento de carrinho espera interagir com o **ServicoDeFrete** genérico, mas em tempo de execução recebe uma instância de **FreteExpresso**, a aplicação poderá estourar um erro indesejado. Explique qual o princípio S.O.L.I.D. foi violado pela classe **FreteExpresso** e porque essa arquitetura é considerada frágil.

7. Em um sistema de Recursos Humanos (RH), um desenvolvedor precisa notificar o gerente responsável por um funcionário. Para obter o e-mail do gerente, ele escreveu a seguinte linha de código:

```
email_gerente = funcionario.get_departamento().get_gerente().get_contato().get_email()
```

- a. Qual é o princípio de projeto que está sendo violada por essa longa cadeia?
 - b. Explique, teoricamente, por que essa estrutura de chamadas prejudica a manutenibilidade do código e proponha uma resolução por meio de um único método na classe **Funcionario** que resolveria o problema de alto acoplamento.
8. Você está projetando o módulo de pagamentos de um *e-commerce*. O fechamento do carrinho exige três etapas obrigatórias e em ordem estrita:
1. Autenticar o usuário;
 2. Processar o débito financeiro;
 3. Enviar o recibo por e-mail.

As etapas 1 e 3 são imutáveis, mas a etapa 2 varia de acordo com o meio de pagamento (PIX, cartão ou boleto). Analisando exclusivamente os padrões comportamentais **Strategy** e **Template Method**, qual deles é a escolha arquitetural correta para resolver este cenário específico? Justifique, focando em como esse padrão lida com o esqueleto do algoritmo e como as subclasses atuam.

9. Em um sistema de monitoramento de infraestrutura, diversos módulos (*log*, *SMS*, *dashboard*) precisam ser notificados quando a CPU do servidor ultrapassar 90%. Para evitar gargalos e desperdício de memória, a equipe técnica definiu que a aplicação pode ter no máximo uma única instância da classe **GerenciadorDeAlertas** rodando em todo o ciclo de vida do software. Explique como os padrões **Observer** e **Singleton** atuam em conjunto na classe **GerenciadorDeAlertas** para atender a todos esses requisitos (especifique a responsabilidade exata de cada um dos padrões).